

1) Prove that 3-SAT Problem is NP-Complete The 3-SAT problem is a Boolean satisfiability problem in which the Boolean expression is written in Conjunctive Normal Form (CNF) and each clause contains exactly three literals. The goal is to determine whether there exists an assignment of truth values to variables such that the complete formula evaluates to true. To prove that 3-SAT is NP-Complete, two conditions must be satisfied. First, the problem must belong to NP. Given a truth assignment, we can easily verify whether all clauses are satisfied by checking each clause in polynomial time. Therefore, 3-SAT belongs to NP. Second, the problem must be NP-Hard. The SAT problem is already known to be NP-Complete. Any SAT instance can be converted into an equivalent 3-SAT instance by breaking clauses with more than three literals into smaller clauses using additional variables. This conversion can be done in polynomial time and does not change satisfiability. Since 3-SAT is in NP and SAT can be reduced to 3-SAT in polynomial time, the 3-SAT problem is NP-Hard. Hence, the 3-SAT problem is NP-Complete.

2) Greedy Knapsack Algorithm The Greedy Knapsack algorithm is used to solve the Fractional Knapsack problem, where items can be divided into fractions. The objective is to maximize total profit without exceeding the capacity of the knapsack. In this approach, items are selected based on a greedy strategy. First, the profit-to-weight ratio of each item is calculated. Items are then sorted in decreasing order of this ratio. The item with the highest ratio is selected first. If the full item fits in the knapsack, it is taken completely. If it does not fit, only a fraction of the item is taken to fill the remaining capacity. This process continues until the knapsack becomes full. The Greedy Knapsack algorithm gives an optimal solution for the fractional knapsack problem but not for the 0/1 knapsack problem. The time complexity of this algorithm is $O(n \log n)$ due to sorting.

3) Rabin-Karp String Matching Algorithm The Rabin-Karp algorithm is a string matching algorithm that uses hashing to search for a pattern within a given text. It compares the hash value of the pattern with the hash values of substrings of the text. First, the hash value of the pattern is computed. Then the hash value of the first substring of the text is calculated. The pattern is slid over the text one character at a time and hash values are compared. If the hash values match, a character-by-character comparison is performed to confirm the match because hash collisions may occur. A rolling hash technique is used to efficiently update hash values. The average and best-case time complexity of Rabin-Karp is $O(n + m)$, while the worst-case complexity is $O(nm)$ due to collisions.

4) Analysis of Multithreaded Algorithms Multithreaded algorithms use multiple threads to perform tasks simultaneously, improving performance by exploiting parallelism. These algorithms are commonly used in multi-core systems. The performance of multithreaded algorithms is analyzed using two measures: Work (T_1) and Span (T_∞). Work is the total time required on a single processor, while span is the time required on infinite processors. Parallelism is defined as T_1 divided by T_∞ and represents the maximum speedup possible. Higher parallelism indicates